

PROYECTO FINAL

SERVICIO DE ENVÍO DE MENSAJES



Sistemas distribuidos

Grupo 80 - Curso 25/26

Nombres, apellidos y NIA	
Alumno 1:	Daniel Santos Méndez (100522786) Grupo 80 - Grado en Ingeniería Informática
Alumno 2:	Sergio Aladro Municio (100522360) Grupo 80 - Grado en Ingeniería Informática

Índice

1. Introducción y objetivos	3
2. Arquitectura general del sistema	3
3. Descripción del código: Servidor en C	4
3.1 Estructura de datos	4
Estructura: Message	4
Estructura: User	4
Gestión de variables globales y concurrencia	4
3.2 Gestión de conexiones concurrentes	5
Ciclo de Vida de la Conexión	5
Funcionamiento del Hilo (client_thread_main)	5
Manejo de Señales del Sistema	5
3.3 Operaciones implementadas	6
3.4 Protocolo de comunicación	7
4. Descripción del código: Cliente en Python	7
4.1 Arquitectura multihilo	7
4.2 Operaciones implementadas	8
4.3 Hilo receptor de mensajes	8
5. Transferencia de ficheros adjuntos	9
Fase 1: Envío y Notificación (A → Servidor → B)	9
Fase 2: Conexión Directa (B → A)	9
Fase 3: Transferencia de Datos	9
6. Servicio Web de normalización	10
7. Servicio RPC de auditoría	10
Interfaz RPC (rpc_service.x)	10
Servidor RPC (rpc_server.c)	11
Integración en el servidor de mensajería	11
8. Compilación y despliegue	11
Requisitos previos	11
Compilación	12
Despliegue y orden de ejecución	12
9. Batería de pruebas	12
10. Conclusiones y problemas encontrados	14

1. Introducción y objetivos

El objetivo de esta práctica es diseñar e implementar un sistema distribuido de mensajería entre usuarios, funcionalmente similar al de una versión simplificada de WhatsApp. El sistema se compone de varios procesos que se comunican mediante distintas tecnologías: **sockets TCP** para la comunicación entre cliente y servidor, **RPC (ONC-RPC)** para el servicio de auditoría de operaciones, y un **servicio web REST** desarrollado en Flask para la normalización de mensajes.

El sistema permite a los usuarios registrarse, conectarse, enviar mensajes de texto (y ficheros adjuntos), desconectarse y consultar usuarios activos. Los mensajes dirigidos a usuarios desconectados se almacenan en el servidor y se entregan en cuanto el destinatario se conecta.

La implementación se divide en:

- **Servidor:** programa en C, concurrente y multihilo, que gestiona el estado global del sistema.
- **Cliente:** programa en Python, también multihilo, que ofrece una interfaz interactiva por consola.
- **Servicio web:** servidor Flask en Python para normalizar mensajes antes de enviarlos.
- **Servidor RPC:** programa en C que actúa como servicio de auditoría, registrando cada operación realizada en el sistema.

2. Arquitectura general del sistema

El sistema sigue una arquitectura **cliente-servidor** donde múltiples clientes se conectan a un único servidor de mensajería central. Cada vez que un cliente desea realizar una operación, establece una **nueva conexión TCP** con el servidor, envía la operación codificada como cadenas de caracteres terminadas en \0, y cierra la conexión al finalizar (protocolo sin estado por operación).

Los componentes son:

- **Servidor de mensajería (C):** escucha en un puerto configurable. Por cada conexión entrante lanza un hilo nuevo. Mantiene en memoria la lista de usuarios registrados (nombre, estado, IP, puerto, mensajes pendientes) y un contador de identificadores de mensaje por usuario.
- **Cliente (Python):** tiene dos hilos. El hilo principal gestiona la entrada del usuario por consola y envía peticiones al servidor. El hilo secundario (creado en CONNECT) escucha en un puerto local para recibir mensajes del servidor destinados a este cliente.
- **Servicio web (Python/Flask):** recibe mensajes en texto plano vía HTTP POST y los devuelve normalizados (sin espacios redundantes). El cliente lo consulta antes de enviar cualquier mensaje.

- **Servidor RPC (C, ONC-RPC):** el servidor de mensajería actúa como cliente RPC y notifica a este servidor cada operación que recibe, indicando usuario, operación y (si aplica) nombre de fichero.

Es importante destacar que en la **transferencia de ficheros la comunicación es directa entre clientes** (peer-to-peer mediante sockets TCP), sin pasar por el servidor de mensajería.

3. Descripción del código: Servidor en C

3.1 Estructura de datos

El servidor mantiene en memoria dos estructuras principales definidas en **server.c**.

Estructura: Message

Representa un mensaje pendiente de entrega y contiene los siguientes campos:

- **sender:** puntero a cadena dinámica con el nombre del remitente.
- **id:** identificador numérico de tipo unsigned int.
- **text:** puntero a cadena dinámica con el texto del mensaje.
- **filename:** puntero a cadena dinámica con el nombre del fichero adjunto, NULL si no hay adjunto.
- **next:** puntero al siguiente mensaje en la cola enlazada.

Estructura: User

Representa a un usuario registrado y contiene:

- **username:** puntero a cadena dinámica.
- **connected:** entero, 0 si desconectado y 1 si conectado.
- **ip:** array char[INET_ADDRSTRLEN] con la dirección IP del cliente cuando está conectado.
- **port:** entero con el puerto del hilo receptor del cliente.
- **last_message_id:** último identificador de mensaje asignado a este usuario como remitente, de tipo unsigned int.
- **pending_head:** puntero al primer mensaje pendiente de la cola.
- **pending_tail:** puntero al último mensaje pendiente de la cola.
- **next:** puntero al siguiente usuario en la lista enlazada.

Gestión de variables globales y concurrencia

- **Lista Global:** La lista global **g_users** es una lista enlazada simple de estructuras **User**.
- **Sincronización:** Protegida por el mutex estático **g_users_mutex** de tipo **pthread_mutex_t**.
- **Secciones Críticas:** Toda función que accede o modifica **g_users** lo hace dentro de una sección crítica delimitada por **pthread_mutex_lock** y **pthread_mutex_unlock**.
- **Funciones Especiales:** Las funciones auxiliares marcadas con el sufijo **_locked** asumen que el mutex ya está tomado por el llamador.

Gestión de Memoria: Toda la gestión de memoria dinámica se hace con **malloc**, **calloc**, **realloc** y **free**. La función auxiliar **dup_string** realiza copias profundas de cadenas. Las funciones **free_message**, **free_all_messages** y **free_user** liberan de forma recursiva cada estructura cuando ya no es necesaria, evitando fugas de memoria.

3.2 Gestión de conexiones concurrentes

Ciclo de Vida de la Conexión

1. **Escucha y Aceptación:** Crea un socket **TCP** en el puerto indicado por línea de comandos y entra en un bucle de **accept()**.
2. **Preparación de Argumentos:** Por cada conexión aceptada se reserva dinámicamente una estructura **ClientThreadArgs** que contiene:
 - El descriptor del socket del cliente (**client_fd**).
 - La IP remota (**peer_ip**).
3. **Ejecución en Paralelo:**
 - Se lanza un hilo con **pthread_create** ejecutando la función **client_thread_main**.
 - El hilo se marca inmediatamente como **detached** con **pthread_detach()** para que sus recursos se liberen automáticamente al terminar, sin necesidad de llamar a **pthread_join()**.
4. **Disponibilidad:** De esta forma el bucle principal del servidor queda libre para aceptar nuevas conexiones sin esperar a ningún hilo.

Funcionamiento del Hilo (**client_thread_main**)

Una vez en ejecución, el hilo realiza las siguientes tareas:

- **Recepción:** Recibe los argumentos y libera la estructura auxiliar.
- **Lectura:** Lee la cadena de operación con **recv_cstring**.
- **Despacho:** Deriva la llamada a la función correspondiente (**handle_register_request**, **handle_connect_request**, etc.).
- **Finalización:** Al terminar cierra el descriptor del cliente con **close** y finaliza el hilo con **pthread_exit**.

Manejo de Señales del Sistema

Para garantizar la estabilidad y el cierre controlado, se configuran las siguientes señales:

- **SIGINT:** Se registra un manejador mediante `sigaction` que:
 - Pone la variable global **g_running** a 0.
 - Cierra el socket principal, provocando que el `accept` devuelva un error y el bucle termine limpiamente.
- **SIGPIPE:** Se ignora mediante `signal(SIGPIPE, SIG_IGN)` para evitar que el proceso finalice inesperadamente al intentar escribir en un socket que el cliente ha cerrado.

3.3 Operaciones implementadas

- **REGISTER:** la función `handle_register_request` lee el nombre de usuario con `recv_cstring`. Con el mutex tomado, comprueba si ya existe un usuario con ese nombre llamando a `find_user_locked`. Si no existe, reserva una nueva estructura `User`, inicializa todos sus campos (estado desconectado, IP vacía, puerto 0, `last_message_id` a 0, cola de mensajes vacía) y la inserta al principio de `g_users`. Devuelve código 0 si todo va bien, 1 si el nombre ya estaba en uso. Muestra por consola `s> REGISTER <userName> OK o FAIL`.
- **UNREGISTER:** la función `handle_unregister_request` recorre la lista enlazada buscando el usuario, lo extrae desenlazándolo de la lista y libera toda su memoria (incluidos los mensajes pendientes) fuera del mutex. Devuelve 0 si se borró correctamente, 1 si no existía.
- **CONNECT:** la función `handle_connect_request` recibe nombre de usuario y puerto como cadenas. Con el mutex tomado, verifica que el usuario existe, que está desconectado y que el puerto es un entero válido. Si todo es correcto, copia la IP obtenida del `accept` (parámetro `peer_ip`) y el puerto convertido a entero en los campos del usuario, y cambia `connected` a 1. Devuelve 0 en éxito, 1 si no existe, 2 si ya estaba conectado y 3 en cualquier otro error. Tras enviar la respuesta y mostrar el mensaje de consola, llama a `deliver_pending_messages_for_user` para entregar inmediatamente todos los mensajes que pudieran estar almacenados.
- **DISCONNECT:** la función `handle_disconnect_request` verifica adicionalmente que la IP desde la que llega la petición coincide con `user->ip`, lo que impide que un proceso externo desconecte a otro usuario. Si la verificación pasa, pone `connected` a 0 y borra IP y puerto. Devuelve 0 en éxito, 1 si no existe, 2 si no estaba conectado y 3 en otro error.
- **SEND:** la función `handle_send_request` verifica que tanto remitente como destinatario existen y que el remitente está conectado y su IP coincide con la de la conexión entrante. Si todo es correcto, calcula el siguiente identificador con `next_message_id_locked`, que incrementa `last_message_id` y, si tras el incremento el valor es 0 (desbordamiento natural de unsigned int), lo fija manualmente a 1. El mensaje se añade a la cola del destinatario con `append_pending_message_locked` pasando NULL como nombre de fichero. Se responde al cliente con código 0 y el identificador en forma de cadena. Si el destinatario está conectado se llama a `deliver_pending_messages_for_user`; si no, se muestra por consola `s> MESSAGE <id> FROM <s> TO <r> STORED`.

- **SENDATTACH**: la función **handle_sendattach_request** es idéntica a **SEND** salvo que lee un campo adicional con el nombre del fichero y lo pasa a **append_pending_message_locked**. Al entregar usa **SEND MESSAGE ATTACH** y **SEND MESS ATTACH ACK** en lugar de sus equivalentes sin adjunto.
- **USERS**: la función **handle_users_request** verifica que el solicitante está registrado, conectado y que su IP coincide. Con el mutex tomado llama a **copy_connected_users_locked**, que construye un array dinámico de cadenas en formato **usuario::IP::puerto**. Envía primero el código 0, luego el número de usuarios como cadena y finalmente cada cadena del array. Libera el array tras el envío.
- La función **deliver_pending_messages_for_user** opera en bucle: en cada iteración toma el primer mensaje de la cola, lo copia fuera del mutex, intenta entregarlo llamando a **send_message_to_listener**, y si tiene éxito envía el **ACK** al remitente y elimina el mensaje de la cola. Si la conexión al receptor falla, llama a **mark_user_disconnected_by_name** y abandona el bucle.

3.4 Protocolo de comunicación

Todo el protocolo usa cadenas C terminadas en **\0**. Cada operación implica una conexión TCP nueva que se cierra al finalizar. Los códigos de respuesta se envían como un único byte con la función **send_code**. Los identificadores de mensaje y contadores numéricos se envían como cadenas de caracteres: por ejemplo, "132" para el mensaje con número 132. El envío de mensajes del servidor al cliente se realiza conectando el servidor al hilo receptor del cliente mediante la función **connect_to_client_listener**, que usa **inet_pton** para convertir la IP almacenada y **connect** para establecer la conexión.

4. Descripción del código: Cliente en Python

4.1 Arquitectura multihilo

El cliente está implementado como una clase Python **client** cuyos métodos son todos **@staticmethod**. El estado compartido se almacena como atributos de clase: **_server** y **_port** contienen la dirección del servidor de mensajería, **_current_user** guarda el nombre del usuario actualmente conectado (o **None** si no hay ninguno), **_listener_socket** y **_listener_thread** referencian el socket y el hilo de escucha activos, **_listener_stop_event** es un **threading.Event** que señala al hilo receptor cuándo debe terminar. **_connected_users_cache** es un diccionario que mapea nombres de usuario a tuplas (ip, puerto) obtenidas de la última respuesta **USERS**, y **_print_lock** es un **threading.Lock** que garantiza que las impresiones del hilo de consola y del hilo receptor no se entremezclan.

El hilo principal ejecuta el bucle **shell()**, que lee comandos con **input()**, los parsea y llama al método correspondiente. Cuando el usuario ejecuta **CONNECT**, el método **_start_listener** crea un socket TCP, lo vincula a **("", 0)** para que el sistema operativo asigne un puerto libre, llama a **listen()** y arranca el hilo **_listener_loop** como **daemon**. El puerto asignado se obtiene con **getsockname()[1]** y se envía al servidor como parte del protocolo **CONNECT**.

El socket de escucha tiene un timeout de 0.5 segundos en **accept()** para que el hilo pueda comprobar periódicamente si se ha activado **_listener_stop_event**.

Cuando el usuario ejecuta **DISCONNECT**, el método **_stop_listener** activa el evento de parada y cierra el socket de escucha, lo que provoca que el **accept()** bloqueado devuelva un **OSError** y el hilo termine. Este comportamiento se aplica siempre, tanto si la desconexión con el servidor tiene éxito como si no, tal como exige el enunciado.

4.2 Operaciones implementadas

Todas las operaciones siguen el mismo patrón de comunicación. El método auxiliar **_send_string** codifica el valor en UTF-8, le añade el byte **b"\0"** y llama a **sendall()** para garantizar el envío completo. El método **_recv_string** lee byte a byte hasta encontrar **b"\0"**, con un límite configurable mediante el parámetro **max_size** para evitar consumo excesivo de memoria ante datos malformados. El método **_recv_code** usa **_recv_exact(sock, 1)** y devuelve el valor numérico del único byte recibido.

- **REGISTER / UNREGISTER**: abre conexión TCP al servidor, envía la operación y nombre de usuario, lee el byte de respuesta, cierra la conexión y muestra el mensaje correspondiente en consola.
- **CONNECT**: llama a **_start_listener** para obtener el puerto libre y arrancar el hilo receptor; envía **CONNECT** + nombre + puerto al servidor; lee la respuesta. Si el código no es 0, llama a **_stop_listener** antes de retornar.
- **DISCONNECT**: envía **DISCONNECT** + nombre al servidor, lee la respuesta, y en el bloque finally llama siempre a **_stop_listener** y pone **_current_user** a None.
- **SEND**: llama primero a **_normalize_message** para consultar el servicio web; si el módulo requests no está disponible o el servicio no responde (timeout de 2 segundos), el mensaje se envía sin normalizar. Luego envía **SEND** + remitente + destinatario + mensaje y lee código e identificador.
- **SENDATTACH**: igual que **SEND** pero envía adicionalmente el nombre del fichero. Verifica previamente con **os.path.exists** que el fichero existe localmente.
- **USERS**: envía **USERS** + nombre, lee código, cuenta y cadenas de usuarios en formato **usuario::IP::puerto**, las parsea con **split("::")** y actualiza **_connected_users_cache**.
- **GETFILE**: consulta **_connected_users_cache**; si el usuario no está, refresca llamando internamente a **users()**. Se conecta al hilo receptor del usuario remoto y ejecuta el protocolo **GET FILE**.

4.3 Hilo receptor de mensajes

El método **_listener_loop** acepta conexiones entrantes en bucle y, para cada una, lee la cadena de operación con **_recv_string**. Gestiona cinco tipos de mensajes:

- **SEND MESSAGE**: lee remitente, identificador y texto; muestra s> **MESSAGE** <id> **FROM** <userName>, el texto y **END**.
- **SEND MESS ACK**: lee identificador; muestra c> **SEND MESSAGE** <id> **OK**.
- **SEND MESSAGE ATTACH**: lee remitente, identificador, texto y nombre de fichero; muestra el mensaje seguido de **FILE** <fileName>.
- **SEND MESS ATTACH ACK**: lee identificador y nombre de fichero; muestra c> **SENDATTACH MESSAGE** <id> <fileName> **OK**.
- **GET FILE**: lee el nombre del solicitante y el nombre del fichero pedido. Verifica que el fichero existe con `os.path.exists`. Si existe, envía su tamaño en bytes como cadena y a continuación el contenido en chunks de 4096 bytes con un bucle de lectura. Si no existe, envía "0" como tamaño para señalar el error al solicitante.

Cualquier excepción dentro del bloque de procesamiento de una conexión se captura y descarta para que el hilo siga vivo y pueda atender la siguiente conexión.

5. Transferencia de ficheros adjuntos

La transferencia de ficheros es **directa entre clientes**, sin pasar por el servidor. El flujo completo es el siguiente:

Fase 1: Envío y Notificación (A → Servidor → B)

- **Inicio**: El cliente A ejecuta **SENDATTACH B** "mensaje" /tmp/fichero.txt.
- **Normalización**: El cliente normaliza el mensaje vía servicio web y envía al servidor **SENDATTACH** con el nombre de fichero incluido.
- **Almacenamiento**: El servidor almacena el mensaje junto con el nombre del fichero.
- **Entrega**:
 - Si B está conectado, el servidor lo entrega mediante **SEND MESSAGE ATTACH**.
 - Si no está conectado, lo guarda para entrega posterior.
- **Recepción**: El cliente B recibe el mensaje a través de su hilo receptor y muestra el nombre del fichero adjunto.

Fase 2: Conexión Directa (B → A)

- **Preparación**: El cliente B ejecuta **GETFILE A** /tmp/fichero.txt /tmp/local.txt.
- **Localización**: El cliente busca la **IP y puerto de A** en su tabla local (o refresca con **USERS** si no lo encuentra).
- **Petición**: Se conecta directamente al hilo receptor de A y envía la operación **GET FILE**.

Fase 3: Transferencia de Datos

- **Gestión en A**: El hilo receptor de A gestiona la petición, abre el fichero y realiza el envío en dos partes:

- Envía primero su **tamaño en bytes** como cadena.
 - Envía el **contenido en fragmentos** de 4096 bytes.
- **Gestión en B:**
 - El cliente B recibe primero el tamaño.
 - Recibe los datos en bucle hasta acumular exactamente ese número de bytes.
 - Guarda los datos en el fichero local indicado.
- **Control de Errores:** El tamaño previo permite detectar transferencias incompletas y reportar el error al usuario.

6. Servicio Web de normalización

El servicio web está implementado en Python con el framework **Flask** y expone dos endpoints:

- **POST /normalize:** recibe un **JSON** con el campo **message** y devuelve el mensaje normalizado en el campo **normalized**. La normalización consiste en dividir el texto con `split()` (que elimina cualquier secuencia de espacios, tabuladores y saltos de línea) y reunirlo con `' '.join(...)`, garantizando que las palabras queden separadas por un único espacio.
- **GET /health:** devuelve `{"status": "ok"}` para verificar que el servicio está activo.

El servicio se lanza en **127.0.0.1:5000** por defecto, aunque puerto y host son configurables por línea de comandos (`--host`, `--port`). El cliente Python lo consulta en cada operación **SEND** y **SENDATTACH** antes de enviar el mensaje al servidor, a través del método **_normalize_message**. Este método está diseñado para tolerar fallos: si el módulo `requests` no está instalado o el servicio no responde en 2 segundos, devuelve el mensaje original sin normalizar, permitiendo que el cliente funcione aunque el servicio web no esté disponible.

7. Servicio RPC de auditoría

Interfaz RPC (`rpc_service.x`)

Se ha definido el programa **RPC MSGAUDIT_PROG** (número 0x31227860) con versión **MSGAUDIT_VERS** (1). Expone un único procedimiento: **int LOG_OPERATION(string user, string operation, string filename) = 1;**

Los parámetros son: nombre de usuario, nombre de la operación (REGISTER, CONNECT, etc.) y nombre de fichero (cadena vacía si no aplica). El servidor devuelve 1 en caso de éxito y 0 en caso de error. Esta interfaz se justifica porque agrupa en una sola llamada toda la información necesaria para el registro, minimizando el número de llamadas RPC por operación. El uso de string para todos los parámetros simplifica la serialización XDR y permite extensibilidad futura sin cambiar el número de versión.

Servidor RPC (rpc_server.c)

La función **log_operation_1_svc** recibe el usuario, la operación y el nombre de fichero como cadenas, más un puntero de salida **int *result** donde escribe 1 en caso de éxito. Devuelve **bool_t** según la convención que impone **rpcgen -N -M**. Internamente obtiene la hora actual con **time()** y **localtime()**, formatea un timestamp con **strftime** y muestra por consola una línea con el formato **[timestamp] USER=<user> OPERATION=<op>**. Si el nombre de fichero no es vacío, añade **FILE=<nombre>** al final, cubriendo el caso de **SENDATTACH**. Finaliza con **fflush(stdout)** para garantizar visibilidad inmediata. El fichero incluye también la función **msgaudit_prog_1_freeresult**, que **rpcgen** exige para liberar resultados del programa RPC; en este caso no gestiona memoria dinámica pero debe existir para que el enlace no falle.

Integración en el servidor de mensajería

El servidor de mensajería incluye el fichero de cabecera **rpc_service.h** generado por **rpcgen** y define la variable global **g_rpc_ip** donde almacena la dirección del servidor RPC. Al arrancar, **main** lee la variable de entorno **LOG_RPC_IP** con **getenv** y la copia en **g_rpc_ip**; si la variable no está definida, **g_rpc_ip** queda vacía y las llamadas RPC se omiten silenciosamente.

La función auxiliar **log_rpc_operation(user, operation, filename)** encapsula toda la lógica de la llamada RPC. Lee la variable de entorno **LOG_RPC_IP** directamente con **getenv** en cada invocación; Si no está definida o está vacía, devuelve el control sin realizar ninguna acción. Si está disponible, crea un cliente RPC efímero con **clnt_create** apuntando a esa dirección usando el protocolo TCP. La llamada al procedimiento remoto sigue la convención generada con **rpcgen -N -M**, por lo que **log_operation_1** recibe los tres parámetros de entrada más un puntero al entero de resultado, y devuelve un enum **clnt_stat**. Si el estado no es **RPC_SUCCESS** o el resultado no es 1, el servidor escribe un aviso en **stderr** pero continúa funcionando con normalidad. El cliente RPC se destruye con **clnt_destroy** al finalizar. Si **clnt_create** falla, la función retorna inmediatamente sin interrumpir al servidor de mensajería.

Por último, **log_rpc_operation** se invoca al final del bloque de éxito de cada handler: **en handle_register_request, handle_unregister_request, handle_connect_request, handle_disconnect_request, en deliver_pending_messages_for_user tras cada entrega correcta de un mensaje, y en handle_users_request** cuando la operación es exitosa. En el caso de **SENDATTACH** se pasa el nombre del fichero como tercer argumento; en el resto se pasa cadena vacía.

8. Compilación y despliegue

Requisitos previos

- GCC con soporte para C11 y pthreads.

- Python 3 con Flask (pip install flask) y requests (pip install requests).
- Herramientas ONC-RPC: rpcgen y librería **libtirpc** con sus cabeceras en /usr/include/tirpc. En sistemas Debian/Ubuntu se instala con apt install libtirpc-dev.
- Sistema Linux (probado en las aulas de laboratorio de la UC3M "guernika" y en entornos Docker).

Compilación

- **Generar stubs y compilar todo:** make
- **Compilar solo el servidor RPC:** make rpc-compile

El flag -N -M de rpcgen genera código en modo newstyle con soporte para múltiples argumentos de entrada y parámetros de resultado por puntero. Los flags **-Wno-cast-function-type** y **-Wno-unused-parameter** se aplican únicamente al código generado automáticamente por rpcgen, donde esos patrones son inevitables.

Despliegue y orden de ejecución

Para ejecutar el sistema completo es obligatorio respetar el siguiente orden:

1. **Servidor RPC:** ./rpc_server (anotar su IP).
2. **Servicio web:** python3 web_service.py --host 127.0.0.1 --port 5000 (en cada máquina cliente).
3. **Servidor de mensajería:** export LOG_RPC_IP=<IP_servidor_RPC> y luego ./server -p 8888.
4. **Clientes:** python3 client.py -s <IP_servidor> -p 8888 (uno por usuario).

El sistema funciona con procesos en contenedores Docker con IPs distintas, siempre que los puertos necesarios sean accesibles entre contenedores. Si **LOG_RPC_IP** no se define, el servidor de mensajería arranca igualmente sin auditoría RPC. El detalle de los archivos generados puede consultarse directamente en el Makefile.

9. Batería de pruebas

En esta sección se describen las pruebas realizadas:

Prueba 1 — Registro básico y casos de error

- **Registrar usuario A:** se espera REGISTER OK.
- **Registrar de nuevo usuario A:** se espera USERNAME IN USE.
- **Dar de baja usuario inexistente B:** se espera USER DOES NOT EXIST.
- **Dar de baja usuario A registrado:** se espera UNREGISTER OK.
- **Dar de baja A otra vez:** USER DOES NOT EXIST.

Prueba 2 — Ciclo completo de conexión/desconexión

- Registrar A → conectar A → verificar **CONNECT OK** → desconectar A → **DISCONNECT OK**.
- Intentar conectar un usuario no registrado: **CONNECT FAIL, USER DOES NOT EXIST**.
- Intentar conectar un usuario ya conectado: **USER ALREADY CONNECTED**.
- Intentar desconectar un usuario no conectado: **DISCONNECT FAIL, USER NOT CONNECTED**.

Prueba 3 — Envío y recepción de mensajes con destinatario conectado

- **Registrar y conectar A y B.**
- **A envía mensaje a B:** A ve: SEND OK - MESSAGE 1, B ve: s> MESSAGE 1 FROM A \n <texto> \n END, A ve: c> SEND MESSAGE 1 OK.
- Verificar que el identificador de mensaje se incrementa correctamente con mensajes sucesivos.

Prueba 4 — Mensajes con destinatario desconectado (almacenamiento y entrega diferida)

- **Registrar A y B.** Conectar solo A.
- **A envía mensaje a B:** A ve SEND OK - MESSAGE 1 pero no SEND MESSAGE 1 OK (B no está conectado).
- **Conectar B:** B recibe el mensaje almacenado y A recibe c> SEND MESSAGE 1 OK.
- Enviar varios mensajes mientras B está desconectado y verificar que se entregan en orden al conectarse.

Prueba 5 — Solicitud de usuarios conectados

- Registrar A, B, C. Conectar A y B, no C.
- **A ejecuta USERS:** recibe lista con A y B (2 usuarios), C no aparece.
- **Intentar USERS sin estar conectado:** CONNECTED USERS FAIL, USER IS NOT CONNECTED.

Prueba 6 — Concurrencia: múltiples clientes simultáneos

- Conectar 3 o más clientes simultáneamente al servidor.
- Cruzar mensajes entre ellos y verificar que el servidor los gestiona correctamente sin condiciones de carrera.
- Verificar que el servidor sigue respondiendo a nuevas peticiones mientras procesa otras.

Prueba 7 — SENDATTACH y transferencia de ficheros

- A envía **SENDATTACH B** "mensaje" /tmp/test.txt con un fichero de contenido conocido.
- B recibe el mensaje con el nombre del fichero.
- B ejecuta **GETFILE A** /tmp/test.txt /tmp/recibido.txt.
- Verificar que el contenido de /tmp/recibido.txt coincide con el original.

- Probar con fichero de tamaño grande (> 4096 bytes) para verificar la transferencia por fragmentos.

Prueba 8 — Servicio web de normalización

- Enviar mensaje con espacios múltiples: "Hola mundo" → el servidor recibe "Hola mundo".
- Verificar mediante **SEND** que el mensaje almacenado y entregado está normalizado.

Prueba 9 — Servicio RPC de auditoría

- Con el servidor RPC activo, realizar varias operaciones (**REGISTER**, **CONNECT**, **SEND**, etc.) y verificar que el servidor RPC las imprime con el usuario, operación y timestamp correctos.
- Verificar que **SENDATTACH** incluye el nombre del fichero en el registro.

Prueba 10 — Casos extremos

- Enviar mensaje al límite de 255 caracteres: se entrega correctamente.
- Dar de baja un usuario con mensajes pendientes y verificar que los mensajes se eliminan.
- Servidor RPC caído: el servidor de mensajería continúa funcionando.
- Cliente cierra la conexión abruptamente (kill): el servidor detecta el error y marca al usuario como desconectado.

10. Conclusiones y problemas encontrados

En esta sección final se resumen los principales problemas encontrados durante la implementación y las conclusiones obtenidas tras el desarrollo de la práctica.

- La práctica ha permitido consolidar el uso de sockets TCP en C y Python, comprendiendo la gestión de conexiones concurrentes mediante hilos y la sincronización con mutex.
- La integración de tres tecnologías distintas (sockets, servicios web, RPC) en un mismo sistema ha supuesto un reto de coordinación, especialmente en el orden de arranque de los procesos y la gestión de errores cuando alguno no está disponible.
- Entre los principales problemas encontrados estuvieron la sincronización entre hilos, la gestión correcta del cierre de sockets y el control del overflow del contador de mensajes.
- El desarrollo en entornos Docker ayudó a detectar problemas de red que no aparecen en ejecuciones locales.

Uno de los principales problemas encontrados fue que el servidor de mensajería compilaba y funcionaba correctamente de forma aislada, pero las operaciones no aparecían registradas en el servidor RPC. La causa fue que las llamadas a **log_operation_1** no estaban siendo realizadas desde **server.c**: aunque el servidor RPC estaba correctamente implementado e iniciado, el servidor de mensajería no incluía **rpc_service.h** ni invocaba

notify_rpc en ningún handler. La solución consistió en añadir la función auxiliar `log_rpc_operation`, leer **LOG_RPC_IP** al arrancar con `getenv`, e insertar la llamada a **notify_rpc** al inicio de cada handler. Se comprobó también que era necesario enlazar con la librería **-ltirpc** y compilar junto con **rpc_service_clnt.c** y **rpc_service_xdr.c** era necesario para que el ejecutable final tuviera los stubs del cliente RPC disponibles.

Otro punto que requirió atención fue la sincronización entre el hilo principal y el hilo receptor del cliente Python. El uso de **_print_lock** como mutex de impresión y de **_listener_stop_event** como señal de parada resultó fundamental para evitar salidas por consola entremezcladas y para garantizar que el hilo receptor terminase limpiamente en todos los casos, incluso cuando la desconexión del servidor fallaba.

La transferencia peer-to-peer de ficheros requirió especial cuidado en el protocolo de fin de transferencia. Se optó por enviar primero el tamaño del fichero como cadena, lo que permite al receptor saber exactamente cuántos bytes esperar y detectar transferencias incompletas, frente a la alternativa de usar el cierre de socket como señal de EOF, que resulta menos robusta ante errores de red.

El desarrollo y las pruebas en entornos Docker con IPs distintas permitieron detectar varios problemas que no aparecen en ejecución local, principalmente relacionados con la obtención correcta de la IP del cliente en el servidor (usando la IP del accept en lugar de la que envía el cliente) y con la accesibilidad de puertos entre contenedores.